

Code-Layout, Readability and Reusability

Date	1 October 2026
Team ID	SWTID-2026-4934
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document reviews how well the Incident client-side solution is organized, named and reusable. Because it is built mainly by configuration in ServiceNow, with three short client scripts, the checklist is applied to the High Impact Control UI Policy, its actions and the three client scripts.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	All three client scripts use the same indentation with aligned braces, as shown in the Appendix of the project report.
2	Proper File Structure	Files and folders are logically organized	Yes	The UI Policy, its two actions and the three client scripts are kept as separate records on the Incident table, each with its own name.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	No custom variables were needed. The scripts use the standard parameters (oldValue, newValue, isLoading) and clear field names (impact, urgency, assigned_to).
4	Function / Method Names	Functions are descriptively named	Yes	Functions are named onChange, onSubmit and onCellEdit as the platform requires; the script records carry descriptive names such as "Prevent save if assigned to missing".
5	Code Comments	Inline and block comments explain logic	Partial	The onChange and onCellEdit scripts have short comments, but the onSubmit script has none and the Description field of all three scripts is empty.
6	Modular Design	Code is split into reusable functions/modules	Yes	Mandatory and read-only behaviour sits in the UI Policy; setting Urgency, save validation and list-edit blocking each have their own script.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Each script does one job with no unused variables or dead code. Urgency is set by the onChange script and locked by the UI Policy on purpose, as the two do different things.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	The onSubmit script shows an error under Assigned to and cancels the save, onChange skips form load and empty values, and onCellEdit cancels the edit. There is no try/catch and no role-based exception.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	High Impact Control UI Policy	ServiceNow UI Policy	The pattern can be copied for other impact or priority levels and other tables	Medium
2	UI Policy Actions (Assignment group, Urgency)	UI Policy Action	Can be added to any UI Policy to make a field mandatory or read-only	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
3	Auto set urgency for high impact	Client Script (onChange), JavaScript with g_form	Pattern for filling a dependent field when another field changes; field names and values need editing	Medium
4	Prevent save if assigned to missing	Client Script (onSubmit), JavaScript with g_form	Any save-time check, by changing the condition and the message	Medium
5	Prevent state change via list edit	Client Script (onCellEdit), JavaScript	Any field that should be edited only from the form, by changing the field	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Separate, clearly named components, each with a single purpose
Readability	4	Short scripts with simple conditions that are easy to follow
Reusability	3	The patterns are reusable, but field names, values and messages are hard-coded in the scripts
Documentation / Comments	3	Script descriptions and more comments can be added
Overall Score	3.5	Good for a first version; comments, role exceptions and configurable messages are the next improvements